

Relazioni tra tabelle

Relazioni tra tabelle

Uno a uno, uno a molti e molti a molti spiegati con esempi concreti.

Quando le tabelle si collegano

Un database diventa davvero utile quando le tabelle non restano isolate.

Pensa a una scuola. Hai studenti, classi, insegnanti e voti. Ogni elenco ha senso da solo, ma il valore vero arriva quando capisci come si collegano.

In SQL questi collegamenti si chiamano **relazioni**.

Le relazioni più comuni sono tre:

- uno a uno
- uno a molti
- molti a molti

Uno a uno

Una relazione uno a uno significa che una riga di una tabella corrisponde a una sola riga di un'altra tabella.

Esempio: un utente può avere un solo profilo esteso.

È come una persona e la sua carta d'identità: una persona ha una carta, e quella carta appartiene a quella persona.

```
CREATE TABLE utenti (  
  id INT PRIMARY KEY,  
  email VARCHAR(150) NOT NULL  
);  
  
CREATE TABLE profili (  
  utente_id INT PRIMARY KEY,  
  bio VARCHAR(500),  
  FOREIGN KEY (utente_id) REFERENCES utenti(id)  
);
```

Qui `profili.utente_id` identifica il profilo e, allo stesso tempo, punta all'utente collegato.

Uno a molti

Questa è una delle relazioni più comuni.

Un cliente può avere molti ordini. Ma ogni ordine appartiene a un solo cliente.

```
CREATE TABLE clienti (  
  id INT PRIMARY KEY,  
  nome VARCHAR(100) NOT NULL  
);  
  
CREATE TABLE ordini (  
  id INT PRIMARY KEY,  
  cliente_id INT NOT NULL,  
  totale DECIMAL(10, 2) NOT NULL,  
  FOREIGN KEY (cliente_id) REFERENCES clienti(id)  
);
```

La colonna `cliente_id` sta nella tabella `ordini`, perché ogni ordine deve ricordare a quale cliente appartiene.

È come una biblioteca: un lettore può fare molti prestiti, ma ogni prestito è intestato a un lettore preciso.

Molti a molti

Una relazione molti a molti significa che entrambe le parti possono collegarsi a più righe.

Per esempio:

- un ordine può contenere molti prodotti
- lo stesso prodotto può comparire in molti ordini

In questi casi si usa una **tabella ponte**.

```
CREATE TABLE righe_ordine (  
  ordine_id INT,  
  prodotto_id INT,  
  quantita INT,  
  PRIMARY KEY (ordine_id, prodotto_id),  
  FOREIGN KEY (ordine_id) REFERENCES ordini(id),  
  FOREIGN KEY (prodotto_id) REFERENCES prodotti(id)  
);
```

`righe_ordine` collega ordini e prodotti. In più salva anche la quantità.

È come un foglio di iscrizioni: non è solo la lista degli studenti e non è solo la lista dei corsi. Dice quale studente partecipa a quale corso.

Come capire che relazione ti serve

Fai la domanda in entrambe le direzioni.

Per esempio:

- un cliente può avere molti ordini?
- un ordine può appartenere a molti clienti?

Se “molti” vale da una parte sola, probabilmente hai una relazione uno a molti.

Se “molti” vale da entrambe le parti, probabilmente ti serve una tabella ponte.

Il ruolo delle chiavi esterne

Una **chiave esterna** (`FOREIGN KEY`) protegge il collegamento tra due tabelle.

Senza questa regola, potresti salvare un ordine con `cliente_id = 999` anche se non esiste nessun cliente con id 999.

Con la chiave esterna, il database controlla per te che il collegamento abbia senso.

Riepilogo rapido

- Uno a uno: una riga corrisponde a una sola riga.
- Uno a molti: una riga può collegarsi a molte righe.
- Molti a molti: di solito serve una tabella ponte.
- Le chiavi esterne aiutano il database a proteggere i collegamenti.